

Lekcja 23

Temat: Wielowątkowość i wieloprocusowość: Threading, multiprocessing, GIL i jego ograniczenia.

W Pythonie wielowątkowość różni się od innych języków ze względu na **Globalny Blok Interpretera (GIL)**. Oto kluczowe informacje:

GIL (Global Interpreter Lock)

- Mechanizm **uniemożliwiający równoczesne wykonywanie kodu Python przez wiele wątków**
- **W danym momencie tylko jeden wątek** wykonuje kod Python
- Chroni przed problemami z pamięcią, ale ogranicza rzeczywistą równoległość

```
import threading
import time

def io_task(name, duration):
    print(f"Zadanie {name} rozpoczęte")
    time.sleep(duration) # Symulacja I/O
    print(f"Zadanie {name} zakończone")

# Tworzenie wątków
thread1 = threading.Thread(target=io_task, args=("A", 2))
thread2 = threading.Thread(target=io_task, args=("B", 1))
...

    • args=("A", 2) - argumenty dla funkcji (krotka)
      ○ "A" → pierwszy argument name w funkcji
      ○ 2 → drugi argument duration w funkcji
    ...

thread1.start()
thread2.start()

# Czekanie na zakończenie
thread1.join()
thread2.join()
```

Wyjasnienie:

```
Główny: startuję wątki
A: start
B: start
Główny: czekam na zakończenie
B: koniec (trwało 1s)
A: koniec (trwało 2s)
Główny: koniec programu
```

Wątki są zarządzane przez system operacyjny - to **nie Python** decyduje który wątek działa kiedy. Możliwe scenariusze:

1. **W2 może skończyć przed W1** (jak wyżej)
2. **W1 może zacząć pierwszy** - różne czasy

Synchronizacja wątków

To sposób kontrolowania dostępu wielu wątków do wspólnych zasobów (np. zmiennych, plików, struktur danych), żeby uniknąć błędów typu „race condition” (wyścig wątków).

Problem bez synchronizacji

Jeśli dwa wątki jednocześnie modyfikują tę samą zmienną, wynik może być nieprzewidywalny.

```
import threading

counter = 0

def increment():
    global counter
    for _ in range(100000):
        counter += 1

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)

t1.start()
t2.start()
```

```
t1.join()
t2.join()
```

```
print(counter) # ❌ wynik często ≠ 200000
```

Lock (mutex – mutual exclusion) to mechanizm, który pozwala tylko jednemu wątkowi na dostęp do danego fragmentu kodu w tym samym czasie. Czyli: *albo jeden wątek wykonuje kod, albo drugi czeka*

```
import threading
```

```
counter = 0
lock = threading.Lock()
```

```
def increment():
    global counter
    for _ in range(100000):
        with lock: # 🗝️ blokada
            counter += 1
```

```
t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)
```

```
t1.start()
t2.start()
```

```
t1.join()
t2.join()
```

```
print(counter) # ✅ zawsze 200000
```

```
# Bez locka wystąpiłby race condition (wyścig wątków)
```

Wieloprocusowość omija ograniczenia GIL poprzez tworzenie oddzielnych procesów, każdy z własnym interpreterem Pythona i przestrzenią pamięci.

Podstawy multiprocessing

Przykład:

Tworzysz **dwa osobne procesy**, które:

- wykonują ciężkie obliczenia (CPU)
- działają **równolegle** (nie jak wątki)

```
from multiprocessing import Process
import os
import time

def cpu_task(name, duration):
    pid = os.getpid() # ID procesu
    print(f"Proces {name} (PID: {pid}) startuje")

    # Intensywne obliczenia CPU
    result = 0
    for i in range(10000000):
        result += i ** 2

    time.sleep(duration)
    print(f"Proces {name} kończy")

if __name__ == "__main__":
    p1 = Process(target=cpu_task, args=("A", 2))
    p2 = Process(target=cpu_task, args=("B", 1))

    p1.start()
    p2.start()

    p1.join()
    p2.join()
```

Tworzenie procesów

1. Klasa Process

```
from multiprocessing import Process

def worker(name):
    print(f"Pracownik: {name}")

if __name__ == "__main__":
```

```

# Pojedynczy proces
p = Process(target=worker, args=("Jan",))
p.start()
p.join()

# Wiele procesów
processes = []
for i in range(4):
    p = Process(target=worker, args=(f"P{i}",))
    processes.append(p)
    p.start()

for p in processes:
    p.join()

```

2. Dziedziczenie po Process

```

from multiprocessing import Process

class CustomProcess(Process):
    def __init__(self, name):
        super().__init__()
        self.name = name

    def run(self):
        print(f"Proces {self.name} działa")
        # Tu kod procesu

if __name__ == "__main__":
    p = CustomProcess("Worker-1")
    p.start()
    p.join()

```

Komunikacja między procesami

Queue (kolejka)

```

from multiprocessing import Process, Queue
import time

def producer(queue):
    for i in range(5):
        queue.put(f"Dane-{i}")

```

```

        print(f"Wyprodukowano: Dane-{i}")
        time.sleep(0.5)
    queue.put(None) # Sygnał końca

def consumer(queue):
    while True:
        item = queue.get()
        if item is None:
            break
        print(f"Skonsumowano: {item}")
        time.sleep(1)

if __name__ == "__main__":
    q = Queue()

    p1 = Process(target=producer, args=(q,))
    p2 = Process(target=consumer, args=(q,))

    p1.start()
    p2.start()

    p1.join()
    p2.join()

```

Pipe (potok)

```

from multiprocessing import Process, Pipe

def sender(conn):
    conn.send("Wiadomość od nadawcy")
    conn.send(["lista", "danych", 123])
    conn.close()

def receiver(conn):
    while True:
        try:
            msg = conn.recv()
            print(f"Otrzymano: {msg}")
        except EOFError:
            break

if __name__ == "__main__":
    parent_conn, child_conn = Pipe()

    p = Process(target=sender, args=(child_conn,))
    p.start()

```

```
receiver(parent_conn)
p.join()
```

Współdzielenie danych

Value i Array (shared memory)

```
from multiprocessing import Process, Value, Array

def worker(val, arr):
    val.value += 10
    for i in range(len(arr)):
        arr[i] *= 2

if __name__ == "__main__":
    # Współdzielone dane
    shared_val = Value('i', 5)      # 'i' = integer
    shared_arr = Array('i', [1, 2, 3, 4]) # tablica intów

    p = Process(target=worker, args=(shared_val, shared_arr))
    p.start()
    p.join()

    print(f"Wartość: {shared_val.value}") # 15
    print(f"Tablica: {shared_arr[:]})"    # [2, 4, 6, 8]
```

Typy dla Value/Array:

- 'i' - signed int
- 'd' - double (float)
- 'f' - float
- 'c' - char
- 'b' - signed char